Bit Manipulation — Complete Guide

Enhanced Beginner Edition — original content preserved, with added plain-English explanations, brute-force reasoning, and "why we optimize" insight for every problem.

151. Introduction to Bit Manipulation

Understanding the Problem NEW

Every number your computer works with is ultimately stored as a sequence of bits (0s and 1s) in binary. Bit manipulation means directly operating on those individual bits — using operators like AND (`&`), OR (`|`), XOR (`^`), NOT (`~`), and shifts (`<<` , `>>`) — instead of working with the number as a whole via normal arithmetic.

Brute Force Approach NEW

There's no 'brute' alternative to introduce here — this section is about learning the vocabulary and behaviour of each bitwise operator, which is foundational for every problem that follows in this chapter.

Why We Optimize NEW

Bitwise operations matter because they let you solve certain problems dramatically faster and with less memory than arithmetic-based approaches — checking or toggling a single bit is O(1), and operations like XOR have almost magical cancellation properties (as you'll see in problems 160 and 163) that let you solve 'find the odd one out' problems without any extra memory at all. This chapter is really about recognizing when a problem is secretly asking a question about individual bits, even if it's not phrased that way on the surface.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Understanding bitwise operators and their applications.

Visual Diagram

```
Bitwise Operators:
       AND (&) : 1 & 1 = 1, else 0
       OR (|)   : 0 | 0 = 0, else 1
       XOR (^) : Same = 0, Different = 1
       NOT (~) : Flips bits
       Left Shift (<<) : x << 1 = x * 2
       Right Shift (>>): x >> 1 = x / 2
```

```
       Example: 5 (0101) & 3 (0011) = 1 (0001)
                5 (0101) | 3 (0011) = 7 (0111)
                5 (0101) ^ 3 (0011) = 6 (0110)
                5 << 1 = 10 (1010)
                5 >> 1 = 2 (0010)

       Binary Representation:
       0 = 0000
       1 = 0001
       2 = 0010
       3 = 0011
       4 = 0100
       5 = 0101
       6 = 0110
       7 = 0111
       8 = 1000
```

Solution Approach

Concept: Understanding binary operations.

Java Implementation

```
class BitManipulation {
           // Bitwise AND
           public int bitwiseAnd(int a, int b) {
               return a & b;
           }

           // Bitwise OR
           public int bitwiseOr(int a, int b) {
               return a | b;
           }

           // Bitwise XOR
           public int bitwiseXor(int a, int b) {
               return a ^ b;
           }

           // Left Shift (multiply by 2)
           public int leftShift(int a, int positions) {

              return a << positions;
          }

          // Right Shift (divide by 2)
          public int rightShift(int a, int positions) {
              return a >> positions;
          }
      }
```

Time & Space Complexity

Time Complexity: O(1) | Space Complexity: O(1)

152. Check if the i-th Bit is Set or Not

Understanding the Problem NEW

Given a number and a bit position i (counting from 0 at the rightmost bit), determine whether that specific bit is a 1 ('set') or a 0 ('unset'). E.g. in binary `1010`, bit 1 (second from the right) is set.

Brute Force Approach NEW

Without bitwise tools, you might convert the number to a binary string (via repeated division by 2) and check the character at the corresponding position — this works but is far more roundabout than necessary.

Why We Optimize NEW

You can check any bit directly with two operators: shift 1 left by i positions (`1 << i`) to create a 'mask' with only that one bit set, then AND (`&`) it with the number — the result is non-zero exactly when that bit was set. This is an O(1) operation using the hardware's native bit operations, completely avoiding any string conversion or division.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Check if the i-th bit (0-indexed) is set in a number.

Visual Diagram

```
Number: 13 (1101)
       Check bit positions:
       bit 0: 1 ✓ set
       bit 1: 0 ✗ not set
       bit 2: 1 ✓ set
       bit 3: 1 ✓ set

       Method: n & (1 << i)
       13 & (1 << 0) = 13 & 1 = 1 ✓
       13 & (1 << 1) = 13 & 2 = 0 ✗
       13 & (1 << 2) = 13 & 4 = 4 ✓
       13 & (1 << 3) = 13 & 8 = 8 ✓
```

```
       Check for bit 2:
       1 << 2 = 4 (0100)
       13 (1101) & 4 (0100) = 4 (0100) != 0 → Set
```

Solution Approach

Concept: Use left shift and AND operator.

Algorithm

```
Algorithm IS_BIT_SET(n, i):
           Return (n & (1 << i)) != 0
```

Java Implementation

```
class Solution {
           public boolean isBitSet(int n, int i) {
               return (n & (1 << i)) != 0;
           }
       }
```

Time & Space Complexity

Time Complexity: O(1) | Space Complexity: O(1)

153. Check if a Number is Odd or Not

Understanding the Problem NEW

Determine if a number is odd or even.

Brute Force Approach NEW

The everyday way: compute `number % 2` and check if it's 1 or 0 — this works and is already quite fast, but a modulo operation is doing more work than strictly necessary.

Why We Optimize NEW

A number is odd exactly when its **very last bit** (the least significant bit) is 1 — this is just what binary place value means (the last bit represents 'is there a 1's place contribution'). So `number & 1` checks that single bit directly, which is typically an even simpler, faster operation for a CPU than a full modulo division — a good first example of how a bitwise trick can replace a basic arithmetic operation with something conceptually (and often practically) cheaper.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Determine if a number is odd using bit manipulation.

Visual Diagram

```
Binary representation:
       Even numbers end with 0:
       2 = 10
       4 = 100
       6 = 110

       Odd numbers end with 1:
       1 = 1
       3 = 11
       5 = 101

       Check: n & 1
```

```
5 (101) & 1 (001) = 1 → Odd ✓
4 (100) & 1 (001) = 0 → Even ✓
```

Solution Approach

Concept: Check the least significant bit.

Algorithm

```
Algorithm IS_ODD(n):
          Return (n & 1) == 1
```

Java Implementation

```
class Solution {
          public boolean isOdd(int n) {
              return (n & 1) == 1;
          }
      }
```

Time & Space Complexity

Time Complexity: O(1) | Space Complexity: O(1)

154. Check if a Number is Power of 2

Understanding the Problem NEW

Determine whether a number is a power of 2 (like 1, 2, 4, 8, 16...).

Brute Force Approach NEW

You could repeatedly divide the number by 2 in a loop, checking that it divides evenly each time, until you reach 1 (power of 2) or hit an odd remainder before reaching 1 (not a power of 2) — O(log n) time, correct but requiring a loop for something that has a slicker one-line trick.

Why We Optimize NEW

Every power of 2 has *exactly one* bit set in binary (e.g. 8 is `1000`). There's a classic bit trick: `n & (n-1)` always clears the lowest set bit of n. If n is a power of 2 (only one bit set), this operation results in exactly 0. So the one-line check `n > 0 && (n & (n-1)) == 0` answers the question in O(1) — no loop needed at all, compared to the O(log n) repeated-division approach.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Check if a number is a power of two.

Visual Diagram

```
Powers of 2:
       1 = 1        (1)
       2 = 10       (2)
       4 = 100      (4)
       8 = 1000     (8)
       16 = 10000   (16)

       Property: n & (n-1) == 0 for powers of 2
       8 (1000) & 7 (0111) = 0 ✓
       16 (10000) & 15 (01111) = 0 ✓

       Not powers of 2:
       6 (0110) & 5 (0101) = 4 (0100) ≠ 0 ✗
       10 (1010) & 9 (1001) = 8 (1000) ≠ 0 ✗
```

```
       Edge case: n == 0 → false
```

Solution Approach

Concept: Check if exactly one bit is set.

Algorithm

```
Algorithm IS_POWER_OF_TWO(n):
           If n <= 0: Return false
           Return (n & (n-1)) == 0
```

Java Implementation

```
class Solution {
           public boolean isPowerOfTwo(int n) {
               if (n <= 0) return false;
               return (n & (n - 1)) == 0;
           }
       }
```

Time & Space Complexity

Time Complexity: O(1) | Space Complexity: O(1)

155. Count the Number of Set Bits

Understanding the Problem NEW

Count how many bits are set to 1 in a number's binary representation (also called its 'popcount' or 'Hamming weight'). E.g. 13 in binary is `1101`, which has 3 set bits.

Brute Force Approach NEW

The straightforward approach: check every bit position one at a time (shift and AND, as in problem 152) across all 32 (or 64) bits of the number, counting how many are set — O(number of bits in the number's type), which for a 32-bit integer is a fixed 32 iterations even if the number itself is small.

Why We Optimize NEW

Brian Kernighan's algorithm is a clever shortcut: repeatedly apply `n = n & (n-1)` (which, as noted in problem 154, clears the lowest set bit each time) and count how many times you can do this before n becomes 0. Instead of always checking a fixed 32 bit positions regardless of the number, this loop runs exactly once per *set* bit — so a number like 8 (only one set bit) finishes in a single iteration instead of 32, making the algorithm's speed proportional to how many 1s are actually present rather than the total bit-width of the number's type.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Count the number of 1s in binary representation.

Visual Diagram

```
Number: 13 (1101)
       Set bits: positions 3, 2, 0 → 3 bits

       Method 1: Brian Kernighan's Algorithm
       13 (1101)
       n & (n-1): 13 & 12 = 12 (1100) → count=1
       n & (n-1): 12 & 11 = 8 (1000) → count=2
       n & (n-1): 8 & 7 = 0 (0000) → count=3

       Method 2: Loop through bits
```

```
        13 (1101) & 1 = 1 → count=1, n>>=1
        6 (0110) & 1 = 0 → count=1, n>>=1
        3 (0011) & 1 = 1 → count=2, n>>=1
        1 (0001) & 1 = 1 → count=3, n>>=1
```

Solution Approach

Concept: Brian Kernighan's algorithm or bit loop.

Algorithm

```
Algorithm COUNT_SET_BITS(n):
           count = 0
           While n > 0:
               n = n & (n-1)
               count++
           Return count
```

Java Implementation

```
class Solution {
           // Brian Kernighan's Algorithm
           public int countSetBits(int n) {
               int count = 0;
               while (n > 0) {
                   n = n & (n - 1);
                   count++;
               }
               return count;
           }
       }
```

Time & Space Complexity

Time Complexity: O(number of set bits) | Space Complexity: O(1)

156. Set/Unset the Rightmost Unset Bit

Understanding the Problem NEW

Given a number, find and set (turn to 1) the rightmost bit that is currently 0 (unset). E.g. `1001` → the rightmost unset bit is at position 1, giving `1011`.

Brute Force Approach NEW

You could scan bit positions one at a time from the right (using the check from problem 152), find the first unset one, then set it with a separate OR operation — two passes conceptually (find, then set), or one pass with a conditional break.

Why We Optimize NEW

There's an elegant single-expression trick: `n | (n + 1)`. Adding 1 to n causes a carry to ripple through all the trailing 1-bits (flipping them to 0) until it hits the first 0-bit, which becomes a 1 — and OR-ing the result back with the original n preserves everything that was already set while incorporating that newly-flipped bit. This does in one arithmetic-plus-bitwise expression what would otherwise take an explicit bit-by-bit scan-and-set.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Set the rightmost unset bit to 1.

Visual Diagram

```
Number: 10 (1010)
       Rightmost unset bit: position 1 (0)
       Set: 10 (1010) → 14 (1110)

       Step-by-step:
       n = 10 (1010)
       ~n = 0101 (bitwise complement)
       n+1 = 1011
       ~(n+1) = 0100
       n | (~(n+1)) = 1010 | 0100 = 1110 = 14

       Simpler: n | (n+1)
       10 | 11 = 1010 | 1011 = 1011 = 11 (not correct)
```

```
Wait, correct method:
n | (n+1) works for rightmost unset bit:
10 (1010) | 11 (1011) = 1011 = 11

Examples:
9 (1001) → 1011 = 11
5 (0101) → 0111 = 7
3 (0011) → 0111 = 7 (all bits set)
```

Solution Approach

Concept: Use n | (n+1).

Algorithm

```
Algorithm SET_RIGHTMOST_UNSET(n):
          Return n | (n + 1)
```

Java Implementation

```
class Solution {
          public int setRightmostUnsetBit(int n) {
              return n | (n + 1);
          }
      }
```

Time & Space Complexity

Time Complexity: O(1) | Space Complexity: O(1)

157. Swap Two Numbers

Understanding the Problem NEW

Swap the values of two variables without using a third temporary variable to hold one of them during the swap.

Brute Force Approach NEW

The everyday way to swap uses a temporary variable: `temp = a; a = b; b = temp;` — perfectly correct and actually the clearest, safest approach in real code; this problem exists purely as a bit-manipulation exercise.

Why We Optimize NEW

Using XOR's self-cancelling property (`x ^ x = 0` and `x ^ 0 = x` , the same property that powers problem 160): `a = a ^ b; b = a ^ b; a = a ^ b;` swaps the two values using no extra variable, because after the first line, `a` holds `original_a ^ original_b` , and XOR-ing that against either original value 'cancels out' that value, leaving the other one. Worth noting as a caveat: this trick has a subtle bug if `a` and `b` are the same memory location (XOR-ing a variable with itself zeroes it out), so in real production code the plain temp-variable swap is usually still preferred for clarity and safety — this problem is really about understanding XOR's properties, not about writing better swap code.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Swap two numbers without using third variable.

Visual Diagram

```
Using XOR:
       a = 5 (0101), b = 3 (0011)

       Step 1: a = a ^ b
       a = 0101 ^ 0011 = 0110 (6)

       Step 2: b = a ^ b
       b = 0110 ^ 0011 = 0101 (5)

       Step 3: a = a ^ b
```

```
        a = 0110 ^ 0101 = 0011 (3)

        Result: a=3, b=5

        Using Addition/Subtraction:
        a = 5, b = 3
        Step 1: a = a + b = 8
        Step 2: b = a - b = 8 - 3 = 5
        Step 3: a = a - b = 8 - 5 = 3
        Result: a=3, b=5
```

Solution Approach

Concept: XOR or addition/subtraction.

Algorithm (XOR)

Algorithm SWAP(a, b):

a = a ^ b

b = a ^ b

a = a ^ b

Return {a, b}

Java Implementation

```
class Solution {
            // Using XOR
            public void swapXOR(int[] arr, int i, int j) {
                arr[i] = arr[i] ^ arr[j];
                arr[j] = arr[i] ^ arr[j];
                arr[i] = arr[i] ^ arr[j];
            }

            // Using addition/subtraction
            public void swapAdd(int[] arr, int i, int j) {
                arr[i] = arr[i] + arr[j];
                arr[j] = arr[i] - arr[j];
                arr[i] = arr[i] - arr[j];
            }
        }
```

Time & Space Complexity

Time Complexity: O(1) | Space Complexity: O(1)

158. Divide Two Integers Without Using Multiplication

Understanding the Problem NEW

Divide two integers and return the integer quotient, without using multiplication, division, or modulo operators directly.

Brute Force Approach NEW

The most literal way without division: repeatedly subtract the divisor from the dividend, counting how many subtractions you can do before the remainder is smaller than the divisor — correct, but if the dividend is much larger than the divisor, this can take an enormous number of subtractions (e.g. dividing a billion by 1 would take a billion steps).

Why We Optimize NEW

Instead of subtracting the divisor one copy at a time, subtract the **largest possible power-of-2 multiple** of the divisor that still fits, using left-shifts to quickly generate those large multiples (`divisor << k` effectively computes `divisor × 2^k` without using multiplication) — subtracting off the biggest chunk you can at each step, then trying progressively smaller powers of 2. This is conceptually similar to how long division works in decimal, but using powers of 2 instead of powers of 10, and it turns a potentially O(dividend/divisor) subtraction count into O(log(dividend)) — since each step at least halves the remaining problem, much like fast exponentiation in problem 168.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Divide two integers using bit manipulation.

Visual Diagram

```
Divide: 10 / 3 = 3
        Binary division:
        10 = 1010, 3 = 0011

        Method: Repeated subtraction with shift
        quotient = 0, temp = 3
```

```
        10 >= 3 → quotient = 1
        temp <<= 1 → 6
        10 >= 6 → quotient = 2 (1+1)
        temp <<= 1 → 12
        10 < 12 → stop

        Alternative bit manipulation:
        10 / 3:
        10 - 3*2 = 4 → quotient += 2
        4 - 3*1 = 1 → quotient += 1
        Result: 3

        Edge cases:
        Positive divisor, positive dividend
        Negative handling
        Overflow handling
```

Solution Approach

Concept: Bit shifting for division.

Algorithm

```
Algorithm DIVIDE(dividend, divisor):
            If divisor == 0: Return MAX_VALUE
            sign = (dividend < 0) ^ (divisor < 0)
            n = abs(dividend), d = abs(divisor)
            quotient = 0

            While n >= d:
                temp = d, multiple = 1
                While n >= (temp << 1):
                    temp <<= 1
                    multiple <<= 1
                n -= temp
                quotient += multiple

            Return sign ? -quotient : quotient
```

Java Implementation

```
class Solution {
            public int divide(int dividend, int divisor) {
                if (divisor == 0) return Integer.MAX_VALUE;
                if (dividend == Integer.MIN_VALUE && divisor == -1) {
                    return Integer.MAX_VALUE;
                }

                 boolean isNegative = (dividend < 0) ^ (divisor < 0);
```

```
            long n = Math.abs((long) dividend);
            long d = Math.abs((long) divisor);

            long quotient = 0;
            while (n >= d) {
                long temp = d;
                long multiple = 1;
                while (n >= (temp << 1)) {
                    temp <<= 1;
                    multiple <<= 1;
                }
                n -= temp;
                quotient += multiple;
            }

            return (int)(isNegative ? -quotient : quotient);
        }
    }
```

Time & Space Complexity

Time Complexity: O(log n) | Space Complexity: O(1)

159. Count Number of Bits to be Flipped to Convert A to B

Understanding the Problem NEW

Given two numbers A and B, count how many bit positions need to be flipped to turn A into B.

Brute Force Approach NEW

You could convert both numbers to binary strings and compare them character by character, counting mismatches — correct, but involves an unnecessary string-conversion step.

Why We Optimize NEW

XOR-ing A and B directly produces a number where every bit that *differs* between them is set to 1, and every bit that's the *same* is 0 (this is literally XOR's definition: 'true when inputs differ'). So the answer is simply the **count of set bits** in `A ^ B` — directly reusing the bit-counting technique from problem 155 (Brian Kernighan's algorithm), turning a two-step 'convert to binary strings and diff' process into a one-line XOR followed by a bit count.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Count how many bits need to be flipped to convert a to b.

Visual Diagram

```
a = 10 (1010), b = 7 (0111)

       XOR: a ^ b = 1010 ^ 0111 = 1101 (13)
       Count set bits in XOR = 3

       Bits to flip: positions where bits differ
       a: 1 0 1 0
       b: 0 1 1 1
          ^ ^   ^
          3 bits differ
```

```
      Method: countSetBits(a ^ b)
```

Solution Approach

Concept: XOR to find differing bits.

Algorithm

```
Algorithm COUNT_FLIPS(a, b):
          diff = a ^ b
          Return countSetBits(diff)
```

Java Implementation

```
class Solution {
          public int countBitsToFlip(int a, int b) {
              int diff = a ^ b;
              return countSetBits(diff);
          }

           private int countSetBits(int n) {
               int count = 0;
               while (n > 0) {
                   n = n & (n - 1);
                   count++;
               }
               return count;
           }
      }
```

Time & Space Complexity

Time Complexity: O(number of set bits) | Space Complexity: O(1)

160. Find the Number that Appears Odd Number of Times

Understanding the Problem NEW

In an array where every number appears an even number of times except one, which appears an odd number of times, find that one number. (This is the same 'Single Number' problem seen in the Arrays chapter, revisited here explicitly as a bit-manipulation technique.)

Brute Force Approach NEW

A hashmap-based frequency count finds the answer in O(n) time but O(n) extra space, tracking every distinct value's count explicitly.

Why We Optimize NEW

XOR every number in the array together: pairs of equal numbers XOR to 0 and vanish (`x ^ x = 0`), and XOR-ing with 0 leaves any value unchanged (`x ^ 0 = x`), so after XOR-ing everything, only the number that appeared an odd number of times survives. This is exactly the same technique as the Arrays-chapter 'Single Number' problem, but placed here to make explicit *why* it works — it's a direct consequence of XOR's self-cancelling algebra, achieving O(n) time and O(1) space, better than the hashmap's O(n) space.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find the number that appears odd number of times.

Visual Diagram

```
Array: [1, 2, 3, 2, 1, 3, 4]
       All appear even times except 4

       XOR property:
       1 ^ 1 = 0
       2 ^ 2 = 0
       3 ^ 3 = 0
       4 ^ 0 = 4
```

```
XOR all numbers:
1 ^ 2 ^ 3 ^ 2 ^ 1 ^ 3 ^ 4 = 4

Result: 4 (appears once → odd times)

Array: [4, 3, 4, 4, 3, 4, 5, 6, 6]
XOR all: 5 (appears once → odd)

Array: [1, 1, 2, 2, 3, 3, 3]
XOR all: 3 (appears three times → odd)
```

Solution Approach

Concept: XOR all numbers, duplicates cancel.

Algorithm

```
Algorithm FIND_ODD_OCCURRENCE(arr):
           result = 0
           For each num in arr:
               result ^= num
           Return result
```

Java Implementation

```
class Solution {
           public int findOddOccurrence(int[] arr) {
               int result = 0;
               for (int num : arr) {
                   result ^= num;
               }
               return result;
           }
       }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

161. Power Set

Understanding the Problem NEW

Generate the power set (every possible subset) of a given array — the same goal as the Recursion chapter's 'Print All Subsequences' problem (133), but this time using bit manipulation instead of recursive include/exclude calls.

Brute Force Approach NEW

The recursive include/exclude approach from problem 133 works fine ($O(2^n)$ time, matching the number of subsets that must exist), but requires building a recursion tree with function call overhead for each branching decision.

Why We Optimize NEW

Since there are exactly 2^n subsets of an n-element array, and each subset can be represented by an n-bit number where **bit i tells you whether element i is included**, you can generate every subset just by counting from 0 to 2^n-1 and checking which bits are set in each count value (using the technique from problem 152) to decide which elements to include. This replaces recursive function calls with a simple loop and bitmask checks — same $O(2^n)$ total time as the recursive version, but often with lower constant-factor overhead since there's no recursive call stack involved, and it directly connects the abstract idea of 'include or exclude each element' to a concrete numeric representation (the bitmask).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Generate all subsets using bit manipulation.

Visual Diagram

```
Array: [1, 2, 3]
       All subsets: 2³ = 8 subsets

       Bit representation:
       0: 000 → []
       1: 001 → [1]
       2: 010 → [2]
```

```
3: 011 → [1,2]
4: 100 → [3]
5: 101 → [1,3]
6: 110 → [2,3]
7: 111 → [1,2,3]

For each i from 0 to (1<<n)-1:
  For each bit j:
    If (i >> j) & 1:
      Add arr[j] to subset

n=3, 1<<3 = 8
i=5 (101):
  j=0: (5>>0)&1=1 → include arr[0]
  j=1: (5>>1)&1=0 → skip arr[1]
  j=2: (5>>2)&1=1 → include arr[2]
  → [1,3]
```

Solution Approach

Concept: Use bitmask to represent subsets.

Algorithm

```
Algorithm POWER_SET(arr):
           n = arr.length
           result = []
           For mask = 0 to (1<<n)-1:
               subset = []
               For i = 0 to n-1:
                   If (mask & (1<<i)) != 0:
                       subset.add(arr[i])
               result.add(subset)
           Return result
```

Java Implementation

```
class Solution {
           public List<List<Integer>> powerSet(int[] arr) {
               int n = arr.length;
               List<List<Integer>> result = new ArrayList<>();

                 for (int mask = 0; mask < (1 << n); mask++) {
                     List<Integer> subset = new ArrayList<>();
                     for (int i = 0; i < n; i++) {
                         if ((mask & (1 << i)) != 0) {
                             subset.add(arr[i]);
                         }

                   }
```

```
                result.add(subset);
            }
            return result;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n * 2^n) | Space Complexity: O(2^n * n)

162. Find XOR of Numbers from L to R

Understanding the Problem NEW

Compute the XOR of every integer from L to R inclusive (e.g. L=3, R=5 → 3^4^5), without looping through and XOR-ing every single number one at a time.

Brute Force Approach NEW

Loop from L to R, XOR-ing each number into a running result — O(R-L) time, which becomes slow if R-L is very large (e.g. billions of numbers).

Why We Optimize NEW

XOR from 1 to N follows a repeating 4-step pattern based on N modulo 4 (it cycles through N, 1, N+1, 0 depending on N mod 4) — a fact you can derive and then use as an O(1) formula. Using this, `XOR(L to R) = XOR(1 to R) ^ XOR(1 to L-1)` (since XOR-ing the same prefix twice cancels it out, similar in spirit to how prefix sums let you compute range sums instantly). This collapses a potentially billions-long loop into a small constant number of O(1) formula evaluations — the same 'prefix technique' idea from the Arrays chapter's prefix-sum problems, just applied to XOR instead of addition.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Calculate XOR of all numbers from L to R inclusive.

Visual Diagram

```
XOR from 0 to n pattern:
       0: 0
       1: 1
       2: 3
       3: 0
       4: 4
       5: 1
       6: 7
       7: 0
       8: 8

       Pattern:
```

```
n % 4 == 0 → n
n % 4 == 1 → 1
n % 4 == 2 → n+1
n % 4 == 3 → 0

Find XOR(L,R) = XOR(0,R) ^ XOR(0,L-1)

Example: XOR(2,5)
XOR(0,5) = 1 (5%4=1 → 1)
XOR(0,1) = 1 (1%4=1 → 1)
XOR(2,5) = 1 ^ 1 = 0
Verify: 2^3^4^5 = 2^3=1, 1^4=5, 5^5=0 ✓
```

Solution Approach

Concept: Use XOR prefix property.

Algorithm

```
Algorithm XOR_0_TO_N(n):
          If n % 4 == 0: Return n
          If n % 4 == 1: Return 1
          If n % 4 == 2: Return n+1
          Return 0

      Algorithm XOR_L_R(L, R):
          Return XOR_0_TO_N(R) ^ XOR_0_TO_N(L-1)
```

Java Implementation

```
class Solution {
          public int xorFrom0ToN(int n) {
              if (n % 4 == 0) return n;
              if (n % 4 == 1) return 1;
              if (n % 4 == 2) return n + 1;
              return 0;
          }

          public int xorLtoR(int L, int R) {
              return xorFrom0ToN(R) ^ xorFrom0ToN(L - 1);
          }
      }
```

Time & Space Complexity

Time Complexity: O(1) | Space Complexity: O(1)

163. Find the Two Numbers Appearing Odd Number of Times

Understanding the Problem NEW

Now two numbers each appear an odd number of times (everything else appears an even number of times) — find *both* of them. A direct extension of problem 160's single-odd-number version.

Brute Force Approach NEW

A hashmap frequency count finds both numbers in O(n) time, O(n) space — correct, but this problem exists specifically to explore how far the XOR trick can be pushed with a clever twist.

Why We Optimize NEW

First, XOR everything together — this gives you `x ^ y` (the XOR of the two odd-occurring numbers), since every evenly-occurring number cancels out, leaving the combination of the two numbers you're looking for. Since `x` and `y` are different, `x ^ y` must have *at least one* bit set — pick any such set bit, and use it to **partition** the entire array into two groups: numbers with that bit set, and numbers without. Because x and y differ at that bit, they land in different groups, and every pair of evenly-occurring duplicates lands in the *same* group as each other (since duplicates are bit-identical) — so XOR-ing each group separately isolates x in one group and y in the other. This still achieves O(n) time and O(1) space, extending the simple cancellation trick with one extra partitioning step to separate two 'survivors' instead of just one.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find two numbers that appear odd number of times.

Visual Diagram

```
Array: [1, 2, 3, 2, 1, 4, 3, 5]
       Odd appearing: 4, 5

       Step 1: XOR all numbers
```

```
1^2^3^2^1^4^3^5 = 4^5 = 1 (0001)

Step 2: Find rightmost set bit
1 & -1 = 1

Step 3: Partition numbers by this bit
Numbers with bit 0 set:
1, 3, 1, 3, 5 → XOR = 5
Numbers with bit 1 set:
2, 2, 4 → XOR = 4

Result: [4, 5]

Wait, let's recalc:
4^5 = 1 (0001) → rightmost set bit is position 0
Numbers with bit 0 = 1:
1, 3, 1, 3, 5 → xor = 5
Numbers with bit 0 = 0:
2, 2, 4 → xor = 4
```

Solution Approach

Concept: XOR all numbers, find set bit, partition.

Algorithm

```
Algorithm TWO_ODD_OCCURRING(arr):
           xor_all = 0
           For num in arr: xor_all ^= num

            rightmost_bit = xor_all & -xor_all

            x = 0, y = 0
            For num in arr:
                If (num & rightmost_bit) == 0:
                    x ^= num
                Else:
                    y ^= num

            Return {x, y}
```

Java Implementation

```
class Solution {
           public int[] twoOddOccurring(int[] arr) {
               int xorAll = 0;
               for (int num : arr) {
                   xorAll ^= num;
               }
```

```
                // Rightmost set bit

            int rightmostBit = xorAll & -xorAll;

            int x = 0, y = 0;
            for (int num : arr) {
                if ((num & rightmostBit) == 0) {
                    x ^= num;
                } else {
                    y ^= num;
                }
            }
            return new int[]{x, y};
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

164. Print Prime Factors of a Number

Understanding the Problem NEW

Find all prime factors of a number (the prime numbers that multiply together to produce it). E.g. 60 = 2×2×3×5.

Brute Force Approach NEW

Check every integer from 2 up to the number itself to see if it divides evenly, and if so, whether it's prime — testing primality for every candidate divisor individually is expensive and redundant.

Why We Optimize NEW

You don't need to separately test primality at all: starting from the smallest possible factor (2), repeatedly divide the number by it as many times as it evenly divides (each successful division is automatically a prime factor, since you've already removed all smaller factors first), then move to the next candidate divisor. You only need to check candidate divisors up to **√n**, because if n still has a factor larger than √n remaining after removing all smaller ones, that remaining value itself must be prime. This turns an expensive 'check every number for primality' approach into a direct O(√n) factor-extraction process, with primality of each factor guaranteed for free by the order of extraction rather than checked explicitly.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Print all prime factors of a number.

Visual Diagram

```
Number: 12
       Factors: 2, 2, 3
       Prime factors: 2, 3

       Number: 36
       36 = 2 × 2 × 3 × 3
       Prime factors: 2, 3
```

```
Algorithm:
n = 12
i = 2: 12%2==0 → print 2, n=6
i = 2: 6%2==0 → print 2, n=3
i = 3: 3%3==0 → print 3, n=1
Stop

Number: 13 (prime)
i = 2: 13%2 !=0
i = 3: 13%3 !=0
i = sqrt(13) ≈ 3.6
After loop, if n > 1: print n
Result: 13
```

Solution Approach

Concept: Divide by smallest prime factors.

Algorithm

```
Algorithm PRINT_PRIME_FACTORS(n):
           While n % 2 == 0:
               print 2
               n /= 2

            For i = 3 to sqrt(n) step 2:
                While n % i == 0:
                    print i
                    n /= i

            If n > 1: print n
```

Java Implementation

```
class Solution {
           public List<Integer> primeFactors(int n) {
               List<Integer> factors = new ArrayList<>();

                 while (n % 2 == 0) {
                     factors.add(2);
                     n /= 2;
                 }

                 for (int i = 3; i * i <= n; i += 2) {

                   while (n % i == 0) {
                       factors.add(i);
                       n /= i;
                   }
               }
```

```
            if (n > 1) factors.add(n);
            return factors;
        }
    }
```

Time & Space Complexity

Time Complexity: O(√n) | Space Complexity: O(number of factors)

165. All Divisors of a Number

Understanding the Problem NEW

Find every divisor of a number (every integer that divides it evenly). E.g. divisors of 12: 1, 2, 3, 4, 6, 12.

Brute Force Approach NEW

Check every integer from 1 to n to see if it divides evenly — O(n) time, checking many candidates that obviously can't be factors (like checking if 7 divides 12 when you already know 12/7 isn't an integer, but doing this check for every number up to n regardless).

Why We Optimize NEW

Divisors come in **pairs**: if `d` divides n, then `n/d` is also a divisor. This means you only need to check candidate divisors up to **√n** — for each one that divides evenly, you get two divisors at once (`d` and `n/d`), except when d equals √n exactly (a perfect square), where you only count it once. This drops the search from O(n) to O(√n), exploiting the pairing structure instead of checking every candidate individually.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find all divisors of a number.

Visual Diagram

```
Number: 36
       Divisors: 1, 2, 3, 4, 6, 9, 12, 18, 36

       Method: Check up to sqrt(n)
       i = 1: 36%1==0 → add 1 and 36
       i = 2: 36%2==0 → add 2 and 18
       i = 3: 36%3==0 → add 3 and 12
       i = 4: 36%4==0 → add 4 and 9
       i = 5: 36%5!=0
       i = 6: 36%6==0 → add 6 (36/6=6, same)

       Sorted: 1,2,3,4,6,9,12,18,36
```

```
Number: 28
Divisors: 1,2,4,7,14,28
sqrt(28) ≈ 5.29
i=1: 1,28
i=2: 2,14
i=4: 4,7
i=5: no
```

Solution Approach

Concept: Check up to √n.

Algorithm

```
Algorithm ALL_DIVISORS(n):
           divisors = []
           For i = 1 to sqrt(n):
               If n % i == 0:
                   divisors.add(i)
                   If i != n/i:
                       divisors.add(n/i)
           Sort(divisors)
           Return divisors
```

Java Implementation

```
class Solution {
           public List<Integer> allDivisors(int n) {
               List<Integer> divisors = new ArrayList<>();

                 for (int i = 1; i * i <= n; i++) {
                     if (n % i == 0) {
                         divisors.add(i);
                         if (i != n / i) {
                             divisors.add(n / i);
                         }
                     }
                 }

                 Collections.sort(divisors);
                 return divisors;
            }

       }
```

Time & Space Complexity

Time Complexity: O(√n) | Space Complexity: O(number of divisors)

166. Sieve of Eratosthenes

Understanding the Problem NEW

Find all prime numbers up to some limit N — e.g. all primes up to 30.

Brute Force Approach NEW

Check every number from 2 to N individually for primality (e.g. by testing divisibility up to its square root, as in a per-number primality test) — O(N√N) total, since you repeat a full primality check for every single number.

Why We Optimize NEW

The **Sieve of Eratosthenes** flips the approach: instead of asking 'is this number prime?' for each number individually, start with every number marked as 'potentially prime,' and for each prime you find (starting from 2), **cross off all of its multiples** as definitely-not-prime, since any multiple of a prime can't itself be prime. Whatever survives uncrossed by the end must be prime. This achieves roughly O(N log log N) — dramatically better than checking every number separately — because you're marking off composite numbers in bulk based on their prime factors, rather than individually verifying every candidate from scratch.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find all prime numbers up to n.

Visual Diagram

```
n = 20

       Initialize: [2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20]

       Step 1: 2 is prime → mark multiples of 2
       [2,3,5,7,9,11,13,15,17,19]

       Step 2: 3 is prime → mark multiples of 3
       [2,3,5,7,11,13,17,19]

       Step 3: 5 is prime → mark multiples of 5
```

```
       [2,3,5,7,11,13,17,19]

       Continue up to sqrt(20) ≈ 4.47

       Primes: 2,3,5,7,11,13,17,19

       Array representation:
       Index: 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20
       Value: F F T T F T F T F F F T F T F F F T F T F
```

Solution Approach

Concept: Mark multiples of each prime.

Algorithm

```
Algorithm SIEVE(n):
           isPrime = boolean[n+1]
           Arrays.fill(isPrime, true)
           isPrime[0] = isPrime[1] = false

           For i = 2 to sqrt(n):
               If isPrime[i]:
                   For j = i*i to n step i:
                       isPrime[j] = false

           primes = []
           For i = 2 to n:
               If isPrime[i]:
                   primes.add(i)
           Return primes
```

Java Implementation

```
class Solution {
           public List<Integer> sieveOfEratosthenes(int n) {
               boolean[] isPrime = new boolean[n + 1];
               Arrays.fill(isPrime, true);
               isPrime[0] = isPrime[1] = false;

                 for (int i = 2; i * i <= n; i++) {
                     if (isPrime[i]) {
                         for (int j = i * i; j <= n; j += i) {
                             isPrime[j] = false;

                         }
                    }
               }

               List<Integer> primes = new ArrayList<>();
```

```
            for (int i = 2; i <= n; i++) {
                if (isPrime[i]) {
                    primes.add(i);
                }
            }
            return primes;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n log log n) | Space Complexity: O(n)

167. Find Prime Factorisation of a Number

Understanding the Problem NEW

Find the prime factorization of a number, but this time set up to answer this efficiently for *many* different query numbers, not just once.

Brute Force Approach NEW

Running the per-number O(√n) factorization from problem 164 separately for every single query number works fine for one query, but if you have many queries, you're redoing similar work (checking small primes repeatedly) over and over.

Why We Optimize NEW

Precompute a **Smallest Prime Factor (SPF)** table for every number up to some maximum, using a sieve-like process similar to problem 166 (for each prime found, mark it as the smallest prime factor for all of its yet-unmarked multiples). Once this table is built once (in O(N log log N)), factorizing *any* number up to N becomes fast: repeatedly divide by its SPF and look up the SPF of the result, peeling off prime factors in O(log n) per query instead of O(√n) — a valuable trade-off when you expect to answer many factorization queries, paying a one-time upfront cost to make every individual query much cheaper.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find prime factorization of a number using sieve.

Visual Diagram

```
Number: 36
       Using sieve, find smallest prime factor (SPF)

       SPF array for n=36:
       2:2, 3:3, 4:2, 5:5, 6:2, 8:2, 9:3, 10:2, 12:2, 14:2, 15:3, 16:2, 18:2, 20:2,
21:3, 22:2, 24:2,

       Factorization:
       n = 36
       spf[36] = 2 → divide → n=18
```

```
        spf[18] = 2 → divide → n=9
        spf[9] = 3 → divide → n=3
        spf[3] = 3 → divide → n=1
        Result: 2,2,3,3

        Number: 84
        84 → 2 → 42
        42 → 2 → 21
        21 → 3 → 7
        7 → 7 → 1
        Result: 2,2,3,7
```

Solution Approach

Concept: Precompute smallest prime factor.

Algorithm

```
Algorithm SPF_SIEVE(n):
            spf = new int[n+1]
            For i = 0 to n: spf[i] = i
            spf[0] = spf[1] = 0

            For i = 2 to sqrt(n):
                If spf[i] == i:
                    For j = i*i to n step i:
                        If spf[j] == j:
                            spf[j] = i

            Return spf

        Algorithm FACTORIZE(n, spf):
            factors = []
            While n > 1:
                factors.add(spf[n])
                n /= spf[n]
            Return factors
```

Java Implementation

```
class Solution {
            public int[] smallestPrimeFactor(int n) {
                int[] spf = new int[n + 1];
                for (int i = 0; i <= n; i++) {
                    spf[i] = i;
                }

                 spf[0] = spf[1] = 0;

                 for (int i = 2; i * i <= n; i++) {
```

```
                if (spf[i] == i) {
                    for (int j = i * i; j <= n; j += i) {
                        if (spf[j] == j) {
                            spf[j] = i;
                        }
                    }
                }
            }
            return spf;
        }

        public List<Integer> primeFactorization(int n) {
            int[] spf = smallestPrimeFactor(n);
            List<Integer> factors = new ArrayList<>();

            while (n > 1) {
                factors.add(spf[n]);
                n /= spf[n];
            }
            return factors;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n log log n) for sieve, O(log n) for factorization | Space

168. Power(n, x) - Fast Exponentiation

Understanding the Problem NEW

Compute n raised to the power x efficiently — the same problem as Pow(x,n) from the Recursion chapter (problem 127), revisited here to connect it explicitly to the bit-manipulation theme of this chapter.

Brute Force Approach NEW

Multiply n by itself x times in a loop — O(x) multiplications.

Why We Optimize NEW

Exponentiation by squaring, exactly as in problem 127: this time, it's worth explicitly connecting it to bit manipulation — the exponent x, viewed in binary, tells you exactly which powers of n (n^1, n^2, n^4, n^8, ...) need to be multiplied together to build n^x. Each bit of x that's set corresponds to including that particular squared-power of n in the final product. Seen this way, fast exponentiation is really 'read the bits of the exponent and multiply in the corresponding pre-squared powers' — a nice capstone connecting the divide-and-conquer idea from the Recursion chapter directly to the bit-level view this chapter has been building throughout, still achieving O(log x) instead of O(x).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Calculate n^x efficiently using exponentiation by squaring.

Visual Diagram

```
Power: 2^10 = 1024

       Binary method:
       10 in binary = 1010

       2^10 = 2^(8+2) = 2^8 * 2^2 = 256 * 4 = 1024

       Iterative:
       result = 1, base = 2, exp = 10

       exp & 1 = 0 → exp=5, base=4
```

```
exp & 1 = 1 → result=4, exp=2, base=16
exp & 1 = 0 → exp=1, base=256
exp & 1 = 1 → result=1024, exp=0
Stop → result=1024

Power: 3^5 = 243
exp=5 (101)
exp & 1 = 1 → result=3, exp=2, base=9
exp & 1 = 0 → exp=1, base=81
exp & 1 = 1 → result=243, exp=0
Result=243
```

Solution Approach

Concept: Exponentiation by squaring.

Algorithm

```
Algorithm POWER(n, x):
           result = 1
           base = n
           exp = x

           While exp > 0:
               If exp & 1:
                   result *= base
               base *= base
               exp >>= 1

           Return result
```

Java Implementation

```
class Solution {
           // Iterative
           public long power(long n, int x) {
               long result = 1;
               long base = n;
               int exp = x;

                 while (exp > 0) {
                     if ((exp & 1) == 1) {
                         result *= base;
                     }

                   base *= base;
                   exp >>= 1;
               }
               return result;
           }
```

```
        // Recursive
        public long powerRecursive(long n, int x) {
            if (x == 0) return 1;
            if (x == 1) return n;

            long temp = powerRecursive(n, x / 2);
            if (x % 2 == 0) {
                return temp * temp;
            } else {
                return temp * temp * n;
            }
        }

        // Modular exponentiation
        public long modPower(long n, int x, int mod) {
            long result = 1;
            long base = n % mod;
            int exp = x;

            while (exp > 0) {
                if ((exp & 1) == 1) {
                    result = (result * base) % mod;
                }
                base = (base * base) % mod;
                exp >>= 1;
            }
            return result;
        }
    }
```

Time & Space Complexity

Time Complexity: O(log x) | Space Complexity: O(1) iterative, O(log x) recursive

Bit Manipulation Summary

Problem	Type	Time	Space	Key Technique
Bit Basics	Concepts	O(1)	O(1)	Bitwise operators
Check i-th Bit	Basic	O(1)	O(1)	n & (1<<i)
Check Odd	Basic	O(1)	O(1)	n & 1
Power of 2	Basic	O(1)	O(1)	n & (n-1)
Count Set Bits	Basic	O(k)	O(1)	Brian Kernighan
Set Rightmost Unset	Basic	O(1)	O(1)	n | (n+1)
Swap Numbers	Basic	O(1)	O(1)	XOR swap
Divide Integers	Basic	O(log n)	O(1)	Bit shifting
Count Flips	Interview	O(k)	O(1)	XOR + set bits
Odd Occurrence	Interview	O(n)	O(1)	XOR all
Power Set	Interview	$O(n*2^n)$	$O(2^n*n)$	Bitmask
XOR L to R	Interview	O(1)	O(1)	Pattern
Two Odd Occurring	Interview	O(n)	O(1)	XOR + partition
Prime Factors	Advanced	$O(\sqrt{n})$	O(k)	Division
All Divisors	Advanced	$O(\sqrt{n})$	O(k)	Check up to sqrt
Sieve	Advanced	O(n log log n)	O(n)	Marking multiples
Prime Factorization	Advanced	O(n log log n)	O(n)	SPF array
Power	Advanced	O(log n)	O(1)	Fast exponentiation